📈 Take-Home Challenge: MAG7 Interactive Return Viewer

Overview

You're building a simple full-stack app to visualize daily returns of the **MAG7 stocks** (MSFT, AAPL, GOOGL, AMZN, NVDA, META, TSLA) using data from yfinance. The app will let a user select a date range and interactively explore return series in a grid layout. This is meant to test your ability to build clean, modular, full-stack applications that integrate APIs, handle data fetching, and present rich UI interactivity.

🎯 Requirements

Frontend (React)

- **Grid UI**: Create an interactive, responsive grid (1 box per ticker).
- **Each box should**:
    - Show a line chart of daily returns over time.
    - Allow zooming or tooltip inspection.
    - Display basic summary stats: min, max, mean.
- **Date Picker**:
    - Allow the user to select a start_date and end_date.
    - On change, re-fetch data from the backend.
- **Tech**:
    - React 18+

- Charting library of your choice (e.g., Recharts, Plotly, Chart.js)

Backend (Python)

- **Framework**: Use FastAPI (preferred), Flask is fine too.
- **Endpoint**: /returns?start=YYYY-MM-DD&end=YYYY-MM-DD
- **Behavior**:
  - Use yfinance to fetch **daily close prices** for the MAG7 symbols.
  - Compute **daily % returns** for each symbol.
  - Return JSON structured as:

```
{
 "MSFT": [{"date": "2024-01-01", "return": 0.004}, ...],
 ...
}
```

- **Cache results** in memory to avoid excessive requests.

✅ **Bonus Points**

- Responsive grid layout.
- Handle fetch errors gracefully in the UI.

- Summary table of performance across all 7 names.
- Modular Python backend (e.g., separate fetching, caching, endpoint logic).
- Use Python pandas and type annotations.

📂 Submission

Send back:

- A GitHub repo or zip file.
- README with:
    - Setup instructions
    - How to run frontend + backend
    - Any assumptions you made

🧠 What We're Looking For

Area	Entry	Mid-Level
API integration	Static stub	Working yfinance integration
UI interactivity	Static plot	Responsive, interactive, tooltips
Error handling	None	Clear user messages
Code structure	One file	Modular, typed, readable
Test coverage	None	Basic backend tests or typing
Extensibility	Hardcoded	Clean architecture for add-ons